

Hack the Gate

Una plataforma de venta de entradas que cumple sus promesas

Proyecto 1, concurrencia. Cuatro tareas, una especificación.

1 La venta

En 2026 BTS se presentará en Bogotá. El estadio tiene capacidad para unas 36.000 personas. Entre 150.000 y 300.000 personas intentaron abrir la página de venta de entradas en el mismo segundo. Plataformas de boletería como Ticketmaster o Tuboleta, en teoría, cumplen las siguientes promesas o acuerdos:

- A nadie se le vende un asiento que no existe.
- Nadie paga por un asiento para luego descubrir que otra persona lo ocupa.
- El asiento que se mostró al comprador es el mismo que aparece en su entrada.
- Si dos personas hacen clic sobre el mismo asiento en el mismo milisegundo, exactamente una de ellas lo obtiene.

Construirás la infraestructura base para este tipo de plataformas cuatro veces. Las reglas de venta de entradas se escriben una sola vez y nunca cambian. Lo que cambia es el modelo de concurrencia que hay debajo:

1. Hilos y mutex
2. Memoria Transaccional de Software (STM)
3. Procesos Secuenciales Comunicantes (CSP)
4. Actores

La pregunta interesante no es si cada modelo funciona. Es cuánto cuesta cada uno, qué hace fácil, qué hace posible (por ejemplo, formar una cola o la ausencia de ella) y qué hace imposible de manera menos evidente. Al llegar a la cuarta tarea tendrás cuatro respuestas para el mismo problema y podrás decir cuál pondrías frente a una venta real y por qué.

1.1 Qué se reduce de escala y qué no

Nadie puede ejecutar 200.000 compradores reales en un computador portátil, y no se te pedirá hacerlo. Lo que se conserva es la proporción, que es lo que vuelve difícil el problema: el evaluador enviará aproximadamente 40.000 solicitudes que pedirán cerca de 72.000 asientos en un recinto con 36.000. En este caso, la gran mayoría de las solicitudes debe fallar, y debe fallar de forma limpia y rápida, en lugar de quedarse bloqueada o reportar información falsa. Dos elementos que sí tendría una plataforma real se omiten deliberadamente. No hay sala de espera. No hay red, así que la API usa llamadas a funciones de Elixir y no HTTP.

2 El contrato

El evaluador llama a un único módulo, Gate.Venue, que debe declarar @behaviour Gate.API. El behaviour se entrega en lib/gate/api.ex y no debe modificarse. Ninguna otra parte de tu código se invoca desde el exterior.

Una entrada (ticket) es un mapa con las claves :seat_id, :sector, :price_cop y :hold_id. Un recinto (venue) puede ser lo que tu implementación necesite; el evaluador únicamente lo almacena y luego lo pasa de nuevo a tus funciones. Un hold_id es opaco del mismo modo.

2.1 Identificadores de asiento

Un identificador de asiento tiene la forma "<sector>:R<row>:S<seat>", con la fila y el asiento rellenos con ceros hasta tener tres dígitos y ambos contados desde 1. El asiento 34 de la fila 12 de la tribuna occidental

baja es "occidental_baja:R012:S034". El mismo asiento tiene el mismo nombre en cada ejecución y en cada entrega. Se proporcionan Gate.VenueSpec.seat_id/3 y parse_seat_id/1.

2.2 Las reglas, una por una

No hay pruebas públicas. Por lo tanto, esta sección es exacta y constituye todo el contrato. Si consideras que algo aquí es ambiguo, pregunta; la respuesta se comunica a todos. Las siguientes reglas describen los comportamientos definidos en lib/gate/api.ex:

Nunca bloquea. reserve/4 atiende la solicitud inmediatamente o falla. No la pone en cola, no espera a que otra persona cancele y no vuelve a intentarlo por cuenta propia. Un cliente que quiera intentar de nuevo debe hacerlo por sí mismo.

Todo o nada. Una solicitud de seis asientos que solo puede satisfacerse parcialmente falla y deja el sector exactamente como estaba. Nunca se reserva únicamente una parte de un grupo.

Dividir está permitido, a veces. El modo :any puede dividir un grupo entre varias filas del sector. El modo :contiguous no puede hacerlo; todos los asientos deben pertenecer a una sola fila y sus números deben ser consecutivos.

Dos razones de fallo distintas. :sold_out significa que el sector no tiene qty asientos libres en total. En cambio, :no_contiguous_block significa que sí tiene esa cantidad de asientos libres, pero no existe una secuencia de qty asientos libres consecutivos en ninguna fila.

Los identificadores de reserva nunca se reutilizan. No después de que una reserva expire, sea cancelada o sea confirmada. Nunca, dentro de un mismo recinto.

Confirmar dos veces falla. La primera llamada a confirm/2 sobre una reserva vigente vende los asientos. La segunda devuelve {:error, :already_confirmed}.

Cancelar dos veces es seguro. Cancelar con cancel/2 una reserva que ya expiró o que ya fue cancelada devuelve :ok, porque el cliente pidió liberar esos asientos y los asientos están liberados.

Cancelar una reserva confirmada devuelve {:error, :already_confirmed}, porque una venta no puede deshacerse.

Los identificadores desconocidos fallan de forma segura. Un hold id falsificado, o el identificador de una reserva que fue cancelada, devuelve :unknown_hold. Tu implementación no debe fallar ni cerrarse ante un hold id que nunca emitió, sin importar la forma que tenga.

La disponibilidad es solo una indicación. availability/2 es el número de asientos libres en el momento en que responde. No promete nada sobre la siguiente llamada, porque hay otros clientes ejecutándose. No debe contar asientos reservados temporalmente ni vendidos.

No se exige equidad estricta. No existe una promesa de "primero en llegar, primero en ser atendido". Lo que sí se exige es que ningún cliente quede completamente excluido: un cliente que siga haciendo solicitudes debe eventualmente poder progresar mientras otros también tienen éxito.

Los snapshots son un corte consistente. Dentro de un sector, los tres conjuntos de asientos que devuelve snapshot/1 deben describir un único instante de la historia de ese sector. Los conjuntos son disjuntos por pares y, juntos, contienen todos los asientos del sector. Entre sectores, el corte no tiene que corresponder a un único instante global, porque ningún asiento se mueve entre sectores y congelar todo el recinto a la vez es un ejercicio diferente. :revenue es la suma de los precios de los asientos vendidos.

3 El recinto

La especificación del recinto se proporciona en lib/gate/venue_spec.ex, por ejemplo mediante Gate.VenueSpec.el_campin/1. Los nombres de los sectores y los tres niveles de precio son los de la venta real. Las cantidades de filas y asientos se redondean para obtener la capacidad publicada.

El sector más pequeño es el más costoso, y el evaluador dirige la mayor parte del tráfico hacia los sectores más baratos, que es lo que ocurre en una venta real. Tu código debe funcionar para cualquier especificación con esta forma y no únicamente para esta.

4 Reservas temporales y expiración

Una reserva temporal (hold) vive durante `ttl_ms` milisegundos, valor tomado de la especificación del recinto:

- una vez hayan transcurrido `ttl_ms`, la reserva ya no puede confirmarse, y
- sus asientos quedan disponibles para otra persona.

Como el tiempo bajo carga no es exacto, el evaluador nunca prueba exactamente el límite. Una llamada a `confirm/2` realizada claramente dentro del tiempo de vida debe tener éxito. Una confirmación realizada claramente después debe devolver `:expired`. En el intervalo intermedio se acepta cualquiera de las dos respuestas. La prueba estricta se realiza una vez que la carga se ha detenido y todo ha tenido tiempo de pasar por tres tiempos de vida completos; entonces ya no queda ninguna condición de carrera por perder: en ese momento ningún asiento puede seguir reservado temporalmente.

5 Lo que no puedes usar

Construcción	Por qué está prohibida
<code>GenServer</code>	el modelo de actores es precisamente lo que se te pide construir, no simplemente invocar
<code>Agent</code>	una celda mutable compartida es exactamente el tipo de cosa que debes implementar
<code>Task</code>	crear trabajo y recolectar sus resultados forma parte del ejercicio
<code>Supervisor</code> , <code>Registry</code>	corresponde al territorio del proyecto 2
<code>GenStage</code> , <code>Flow</code>	bibliotecas de concurrencia
<code>:ets</code> , <code>:dets</code>	un heap compartido con operaciones atómicas
<code>:mnesia</code>	solo puede usarse para simular STM
<code>:persistent_term</code>	memoria mutable compartida fuera de tus propios procesos
<code>:atomics</code> , <code>:counters</code>	compare-and-swap de hardware; prácticamente medio mutex gratis
<code>:global</code> , <code>:pg</code>	un gestor de bloqueos distribuidos y grupos de procesos

Lo que queda es justamente lo que necesitas: `spawn`, `send`, `receive`, `Process`, `pattern matching` y las estructuras de datos estándar. También está prohibido usar `Process.sleep` como forma de esperar a que ocurra algo. La ausencia de ETS es especialmente importante: significa que Elixir no te proporciona memoria mutable compartida. Cualquier dependencia que quieras usar debe consultarse primero con el equipo docente, aunque probablemente la respuesta sea no.

6 Evaluación

La evaluación es completamente automatizada y vale 100 puntos. El evaluador no se publica. La especificación anterior está completa, por lo que no hay nada en el evaluador que este documento no te haya indicado ya.

Categoría	Pts	Qué evalúa
Funcional	20	¿Sigue el contrato con un cliente a la vez?
Seguridad	35	Bajo una carga alta, ¿la base de datos sigue siendo consistente? Ningún asiento vendido dos veces, ninguna venta inventada, ninguna venta perdida, cada corte consistente, los ingresos coinciden con los asientos vendidos y no queda nada reservado temporalmente cuando termina la venta.
Adversarial	15	Cinco patrones de carga elegidos para romper un diseño que solo es correcto por accidente: todos compiten por el sector más pequeño; grupos peleando por los últimos asientos; reservas tomadas y devueltas sin pausa; todas las reservas abandonadas para que la expiración haga todo el trabajo; y un sector vendido hasta el último asiento.
Liveness	10	¿Ocurrió algo realmente? La venta termina dentro de su presupuesto de tiempo, cada cliente completa su trabajo, ningún cliente queda excluido y los únicos fallos son los permitidos por el contrato.
Escalabilidad	10	¿El trabajo independiente se ejecuta de manera independiente?
Rendimiento	10	Throughput y latencia de cola (tail latency) comparados con un umbral mínimo para tu modelo.

Usar una construcción prohibida significa que la entrega no será evaluada. Es la misma regla que el curso siempre ha aplicado al código que no sigue la especificación.